AI1220 • ASSIGNMENT 2 • DETAILED FINAL SUBMISSION

ColabDoc

A collaborative rich-text document editor with authenticated multi-user access, Yjs-backed realtime editing, streamed AI assistance, version history, export, and a one-command local setup.

REPOSITORY	SUBMISSION TARGET	WORKING BRANCH	API DOCS
blackeyh/colabdoc	**main**	**assignment-2**	**/docs**

Report Scope

- Requirement-to-evidence mapping for the final Assignment 2 submission.
- Backend, frontend, realtime, AI, and persistence architecture.
- Alignment between the delivered repo and the Assignment 1 design.
- Testing evidence, setup details, and known trade-offs.

Final Stack

- Frontend: React, Vite, Tiptap, Yjs
- Backend: FastAPI, SQLAlchemy, WebSockets
- Data: SQLite by default, PostgreSQL-compatible schema
- AI: OpenAI-compatible provider abstraction with null fallback

Companion Documentation

- **README.md** for setup and protocol details
- **DEVIATIONS.md** for intentional scope reductions
- **ARCHITECTURE_ADDENDUM.md** for final architecture alignment
- **DEMO_CHECKLIST.md** for live grading flow

This PDF is intended to be read as a submission report and evidence summary for the shipped repo snapshot.

Prepared for the final Assignment 2 submission.

SECTION 1

Requirement Coverage

The final submission is judged most fairly when the required behaviors are mapped directly to the shipped implementation and supporting evidence.

Requirement and Evidence Matrix

AREA	IMPLEMENTED BEHAVIOR	EVIDENCE	NOTE / LIMITATION
Authentication	Registration, login, JWT access tokens, refresh token exchange.	`backend/routers/auth.py`, `backend/auth.py`, `tests/test_auth.py`	App-managed JWT auth rather than an external provider.
Document CRUD	Create, rename, edit, load, list, delete, and export documents.	`backend/routers/documents.py`, dashboard and editor components, `tests/test_documents.py`	Canonical content is persisted as structured JSON.
Rich Text	Tiptap editor with headings, lists, code blocks, inline formatting, undo/redo.	`EditorTextarea.jsx`, `EditorBar.jsx`	Legacy plain-text documents are normalized via `contentCompat.js`.
Sharing / Roles	`owner`, `editor`, `commenter`, `viewer` with backend ACL enforcement and UI restrictions.	`permissions.py`, `documents.py`, `tests/test_permissions.py`	The brief required at least three roles; the repo keeps four.
Realtime Collaboration	Authenticated WebSockets, Yjs incremental updates, presence, remote carets, selections, typing.	`main.py`, `websocket_manager.py`, `collaboration.js`, `tests/test_websocket.py`	Latest room snapshot is cached in memory for reconnect and late join.
AI Assistant	Streamed generation, cancel, compare, accept, accept edited, reject, document-level history.	`routers/ai.py`, `providers.py`, `AIPanel.jsx`, `test_ai.py`, `AIPanel.test.jsx`	Default local provider is deterministic unless a real provider is configured.
Version History	Save versions, list versions, restore previous content, synchronize restore to collaborators.	`versions.py`, `VersionsPanel.jsx`, `tests/test_versions.py`	Restore is broadcast back into the live collaboration room.
Export	Portable HTML and plain-text export from the persisted canonical document state.	`exporters.py`, export endpoint, `EditorBar.jsx`	Targets readable portable formats rather than office binaries.
API Docs	Interactive FastAPI docs and OpenAPI schema at `/docs`.	`README.md`, router `summary=` and `response_model=` annotations in auth, documents, permissions, versions, and AI routers.	WebSocket collaboration is documented in the README rather than OpenAPI.
Setup / Tests	`./start.sh`, runnable `.env.example`, backend and frontend tests, rehearsal test.	`start.sh`, root docs, `tests/test_submission_rehearsal.py`	`npm install` currently runs on each startup.

How To Read This Report

This document is organized so a grader can move from a requirement to the implementation, then to the supporting evidence.

- Section 1 is the requirement-to-evidence matrix.
- Sections 2 through 6 explain the shipped architecture and feature paths.
- Section 7 covers setup, API docs, and testing evidence.
- Sections 8 and 9 cover Assignment 1 alignment and known trade-offs.

Documentation Scope

- The report describes the shipped PoC rather than a larger intended architecture.
- Intentional deviations are documented explicitly instead of implied.
- Trade-offs are kept in the report so scope reduction is visible.
- Evidence points to code, tests, and supporting docs in the repository.

SECTION 2

Architecture and Repository Structure

The final system is intentionally a single-repo full-stack PoC, but the runtime responsibilities are still clearly separated.

Runtime View

Browser Client
React + Tiptap + Yjs

→

FastAPI Backend
REST + WebSocket Relay

DB + AI Provider
SQLite / Postgres-compatible + OpenAI-compatible

The browser owns editor state and collaboration state; the backend owns auth, permissions, persistence, export, and provider-backed AI.

Repository Layout

- **`backend/`**: routers, models, AI providers, exporters, WebSocket manager.
- **`frontend/`**: React app, Tiptap editor, Yjs logic, sidebar panels, tests.
- **`tests/`**: backend pytest suite.
- **root docs**: README, deviations, architecture addendum, demo checklist, detailed report.

Why Single Repo

- Easier for a grader to clone and run locally.
- Lower demo risk than a more distributed deployment.
- Still preserves clear frontend/backend responsibility boundaries.
- Avoids pretending the repo has services or packages that do not actually exist.

Frontend Role

Renders auth screens, dashboard, editor, AI sidebar, permissions, versions, and live presence using React components centered around `EditorPage.jsx` and `EditorTextarea.jsx`.

Backend Role

Provides REST APIs for auth, documents, permissions, versions, AI, plus the authenticated collaboration WebSocket endpoint and static frontend hosting.

Data Role

Stores canonical document JSON, permissions, versions, and AI interactions in the database while collaboration snapshots stay lightweight and in-memory.

Architectural Positioning

The final system is simpler than a production multi-service platform, but it is internally consistent. This was a deliberate priority after the earlier report-vs-repo mismatch feedback.

SECTION 3

Backend Design and Data Model

The backend is the authoritative layer for auth, authorization, canonical persistence, export, and AI orchestration.

Backend Modules

- `main.py`
- `auth.py`
- `database.py`
- `models.py`
- `websocket_manager.py`
- `exporters.py`
- `routers/auth.py`
- `routers/documents.py`
- `routers/permissions.py`
- `routers/versions.py`
- `routers/ai.py`
- `ai/providers.py`

Backend Responsibilities

- Issue and refresh JWTs.
- Protect document, AI, and version endpoints.
- Persist canonical document JSON.
- Store version snapshots and AI interaction audit trails.
- Generate export files on demand.
- Authenticate and relay WebSocket collaboration messages.

Core Data Entities

ENTITY	PURPOSE	WHY IT MATTERS
User	Stores account identity and password hash.	Needed for protected routes, sharing, and presence.
Document	Stores title, owner, current persisted content, timestamps.	Acts as the canonical source for load, export, versions, and AI context.
Permission	Maps users to document roles.	Drives editor access, AI access, and read-only behavior.
Version	Stores historical snapshots.	Allows safe restore and demo-visible history management.
AIInteraction	Stores selected text, prompt, provider/model, output, status, and user action.	Creates a document-level AI audit trail rather than an ephemeral UI-only feature.

Authentication

Access and refresh tokens are issued by the app itself. Token-type checks prevent using a refresh token where an access token is expected.

Document Persistence

Even though live collaboration runs through Yjs, the backend still stores canonical rich-text JSON for export, version history, and AI.

Permissions

Role checks are enforced on the server, not just hidden in the UI, which protects AI, save, restore, and other write actions.

Export

Export rendering happens on the backend so the downloaded file matches the persisted server state rather than only one client's local editor state.

SECTION 4

Frontend Design and Editor Experience

The frontend is organized so auth, dashboard, editor behavior, and sidebar workflows stay separated but coordinated.

Main Frontend Areas

- `App.jsx` and `main.jsx` for app bootstrapping and routing.
- `LoginPage.jsx` and `RegisterPage.jsx` for auth entry points.
- `Dashboard.jsx` and `NewDocModal.jsx` for document management.
- `EditorPage.jsx` as the orchestration layer for one document session.
- `EditorTextarea.jsx` for Tiptap + Yjs + remote cursor integration.
- Sidebar panels for AI, AI history, permissions, versions, and active users.

API and Session Handling

- `api.js` attaches the access token to requests.
- On `401`, it performs a single refresh flow using the refresh token.
- After a successful refresh, it retries the failed request once.
- The WebSocket hook follows the same session-recovery idea when a token expires.

Dashboard

Lists accessible documents, supports document creation, and provides the main authenticated entry into the editor workflow.

EditorPage

Coordinates loaded document data, collaboration state, export, AI, versions, permissions, and save behavior for a single open document.

EditorTextarea

Binds Tiptap to Yjs, renders collaborator carets and selections, and keeps AI-applied edits inside normal editor undo/redo history.

Rich-Text Capability

- Headings
- Paragraphs
- Ordered and unordered lists
- Code blocks
- Inline emphasis and inline code
- Undo and redo

Compatibility Layer

Earlier documents may still exist in a simple `{text:"..."}` format.

`contentCompat.js` normalizes both the legacy text-only shape and the final Tiptap JSON shape so older data remains readable instead of requiring a destructive migration.

Frontend Design Principle

The UI is split into focused components so AI, versions, permissions, and active-user presence can evolve independently without turning the editor into one monolithic file.

SECTION 5

Realtime Collaboration and Conflict Handling

The final collaboration path is one of the biggest improvements over the earlier PoC because connected edits now merge at the CRDT layer.

1. **Authenticated Join**
   The client opens `/ws/documents/{id}?token=...`. The backend validates the JWT, checks document access, then emits `init` metadata and the current room state.

2. **Yjs Sync**
   Clients exchange `crdt_update` and `crdt_snapshot` messages so concurrent connected edits merge without relying on last-write-wins full-document replacement.

3. **Presence Layer**
   Separate cursor and typing messages drive active-user display, collaborator carets, collaborator selection highlights, and typing indicators.

4. **Durable Persistence**
   The frontend still sends durable canonical content to the backend so versions, export, and AI use the persisted document rather than only one peer's transient collaborative state.

Important Message Types

MESSAGE	PURPOSE
`init`	Seeds the room with title, role, active users, and collaboration state.
`crdt_update`	Incremental Yjs update from another collaborator.
`crdt_snapshot`	Full room snapshot used for late join or reconnect sync.
`sync_request`	Requests peers to replay the latest snapshot.
`cursor`	Broadcasts selection/caret location.
`typing`	Broadcasts activity without changing document content.
`reset`	Broadcasts a version restore to connected collaborators.

What Improved Compared With the Earlier Flow

- Concurrent edits from connected users now merge through Yjs.
- Presence is visually obvious through in-editor carets and highlights.
- Late joiners and reconnecting users can bootstrap from a current snapshot.
- Version restore is synchronized back into the live room.

Authenticated Rooms **CRDT Merge** **Snapshot Replay** **Remote Carets** **Typing Presence**

Important Trade-Off

The backend keeps the latest opaque room snapshot in memory instead of running a dedicated shared Yjs persistence service. This is much stronger than full-document overwrite syncing, but it is still a PoC collaboration backend rather than a horizontally scaled production system.

SECTION 6

AI Assistant Architecture

The AI flow was upgraded from a blocking pattern into a streamed, audited, role-restricted subsystem that behaves like part of the editor rather than a disconnected demo widget.

AI Backend Flow

1. User selects text and chooses an action such as rewrite or summarize.
2. Backend builds a prompt from the action, selected text, and truncated document context.
3. Provider streams output through `text/event-stream`.
4. Frontend renders `delta` chunks progressively.
5. User explicitly accepts, edits and accepts, or rejects the suggestion.
6. The final decision is recorded in `ai_interactions`.

Event Stream Model

- **`meta`**: interaction row created and stream opened.
- **`delta`**: partial generated text chunk.
- **`done`**: stream completed with final suggestion.
- **`error`**: provider failed; partial text is preserved when possible.

Cancel is handled through `AbortController`, and the backend records the interruption state rather than silently dropping the interaction.

Provider Abstraction

`OpenAIProvider` supports any OpenAI-compatible endpoint, while `NullProvider` keeps the full AI UX testable without network access or vendor billing.

Access Control

Only `owner` and `editor` roles can invoke AI. `viewer` and `commenter` are blocked in both the UI and backend route layer.

Audit Trail

The final history stores selected text, generated prompt, provider/model, output, provider status, user action, and final applied text.

AI Safety and Usability Characteristics

CONCERN	FINAL HANDLING
Premature auto-editing	AI output is never auto-applied. The user must explicitly accept or edit-accept.
Read-only misuse	Viewer and commenter roles are blocked in the UI and receive `403` from the backend if they call AI directly.
Long generation feedback	Streaming provides partial output instead of a blank waiting period.
Undoability	Accepted AI insertions remain part of normal editor history and are covered by `frontend/tests/EditorTextarea.test.jsx`.
Traceability	The document-level history panel shows prompt/model/output/status and user resolution.

SECTION 7

Setup, Testing, and Verification

This section documents the local start path, API documentation surface, and the test artifacts that support the main report claims.

Reviewer Run Path

1. Clone the repository.
2. Run `./start.sh`.
3. Open `http://127.0.0.1:8000`.

`start.sh` creates `.venv`, installs backend requirements when needed, runs `npm install`, creates `.env` from `.env.example` when missing, generates a local JWT secret, builds the frontend, and starts the backend.

API Documentation

- FastAPI exposes interactive docs at `/docs`.
- The REST routers use `summary=` and `response_model=` annotations.
- The README documents the WebSocket collaboration protocol separately.
- The docs surface therefore covers both REST and realtime behavior.

Backend Test Areas

FILE	MAIN COVERAGE
`test_auth.py`	register, login, refresh, invalid token scenarios
`test_documents.py`	document CRUD and access behavior
`test_permissions.py`	sharing and role enforcement
`test_versions.py`	save/list/restore versions
`test_ai.py` / `test_ai_unit.py`	AI streaming, ACL, history, provider behavior
`test_websocket.py`	room join, sync, collaboration message handling
`test_submission_rehearsal.py`	grade-style end-to-end path

Frontend Test Areas

AREA	WHAT IS PROVEN
Auth UI	login/register flows behave correctly
Editor bar	toolbar and document actions render correctly
AI panel	streaming behavior, actions, cancel/error flow
AI history	document-level audit display updates correctly
Editor collaboration	remote cursor rendering and collaboration hooks behave correctly
AI undo proof	`EditorTextarea.test.jsx` proves an accepted AI-style insertion can be reverted through `commands.undo()`

Why the Rehearsal Test Matters

`tests/test_submission_rehearsal.py` ties together auth, sharing, AI, export, version restore, and collaboration messaging in one grading-style path. It supports the report's evidence chain, but it is still separate from the live demo itself.

SECTION 8

Alignment With Assignment 1 and Feedback Resolution

The final repo does not try to silently imitate the broader Assignment 1 architecture. It explains what changed, what stayed, and why.

Preserved Product Ideas

- Collaborative document editing.
- Role-based sharing.
- AI-assisted writing workflow.
- Version history and restore.
- Export of the current document.
- Separation between frontend and backend concerns.

Intentional Simplifications

- Local JWT auth instead of Auth0.
- No external email or object-storage integrations.
- Single-repo layout rather than a larger service decomposition.
- FastAPI relay plus in-memory snapshot cache instead of a dedicated Yjs persistence service.
- HTML and TXT export instead of binary office-format generation.

Resolution of Earlier Feedback

EARLIER ISSUE	FINAL RESOLUTION
System context diagram omitted the authentication provider or left it ambiguous.	Final docs explicitly show backend-managed JWT auth and stop implying an external provider that the runtime does not use.
Long-running AI handling was deferred to synchronous flows.	AI is now streamed, cancellable, progressively rendered, logged, and covered by backend/frontend tests.
Repository structure did not match the documented layout.	The final docs describe the actual single-repo layout and reviewer flow instead of a larger imagined structure.
Report mismatched delivered auth and realtime stack.	`DEVIATIONS.md` and `ARCHITECTURE_ADDENDUM.md` now explain the shipped auth and Yjs/FastAPI collaboration model directly.
README configuration exposed secrets or encouraged unsafe setup.	The repo uses `.env.example` as a template and generates a local JWT secret during first-run setup.
Automated tests were missing.	The final repo includes backend tests, frontend tests, a submission rehearsal test, and an AI undo proof.

Reading This Section Together With The Repo

README, deviations, architecture addendum, demo checklist, and this report are intended to be read together. The goal is to describe the shipped repo directly rather than imply a larger architecture that was not delivered in the final PoC.

SECTION 9

Demo Outline and Known Trade-Offs

The repo includes a recommended demo order and a separate list of known trade-offs. Both are included here so the report stays factual about what was implemented and what remains outside scope.

Recommended Live Demo Order

1. **Auth and Protected Access**
   Show login and the authenticated dashboard flow.

2. **Document Editing**
   Create a document and add rich-text content.

3. **Sharing**
   Show editor, viewer, and commenter roles.

4. **Realtime**
   Show live editing, remote caret, selection highlight, and typing presence.

5. **AI**
   Stream a suggestion, accept one, then undo it and open AI history.

6. **Versions and Export**
   Restore a version and export HTML/TXT.

Remaining Trade-Offs

- **WATCH** Tokens remain in `localStorage` for the current PoC.
- **WATCH** Refresh tokens are stateless rather than revocation-backed.
- **WATCH** Room snapshots are cached in memory rather than persisted in a dedicated Yjs service.
- **WATCH** Null AI provider is the default local fallback; a real provider should be configured for the strongest AI demo.
- **DOCUMENTED** These trade-offs are explained openly in `DEVIATIONS.md` rather than hidden.

These trade-offs are retained in the report so scope boundaries are visible rather than implied.